



PYTHON APPLICATION PROGRAMMING

Anitya Gupta

CHANGING ELEMENTS USING

```
# Use for loop to change the elements in list
```

```
squares = ['red', 'yellow', 'green', 'purple', 'blue']
```

```
for i in range(0, 5):
```

```
    print("Before square ", i, 'is', squares[i])
```

```
    squares[i] = 'pink'
```

```
    print("After square ", i, 'is', squares[i])
```

```
Before square 0 is red
```

```
After square 0 is pink
```

```
Before square 1 is yellow
```

```
After square 1 is pink
```

```
Before square 2 is green
```

```
After square 2 is pink
```

```
Before square 3 is purple
```

```
After square 3 is pink
```

```
Before square 4 is blue
```

```
After square 4 is pink
```

WHILE LOOP

- The for loop is used for a controlled flow of repetition.
- However, what if we don't know when we want to stop the loop?
- What if we want to keep executing a code block until a certain condition is met?
- The while loop exists as a tool for repeated execution based on a condition.
- The code block will keep being executed until the given logical condition returns a False Boolean value.

WORKING OF WHILE LOOP

1. First, you specify a condition that the loop will check before each iteration (repetition) of the code block.
2. If the condition is initially true, the code block is executed.
3. After executing the code block, the condition is checked again.
4. If the condition is still true, the code block is executed again.
5. Steps 3 and 4 repeat until the condition becomes false.
6. Once the condition becomes false, the loop stops, and the program continues with the next line of code after the loop.

WHILE LOOP

- In this example, the condition `count <= 10` is checked before each iteration.
- As long as `count` is less than or equal to 10, the code block inside the loop is executed.
- After each iteration, the value of `count` is incremented by 1 using `count += 1`. Once `count` reaches 11, the condition becomes false, and the loop stops.

WHILE LOOP

Key point of While Loop:

1. A while loop repeatedly executes a block of code as long as a given condition is true.
2. It does not have a fixed number of iterations but continues executing until the condition becomes false.
3. The condition is checked before each iteration, and if it's false initially, the code block is skipped entirely.
4. The condition is typically based on a variable or expression that can change during the execution of the loop.
5. It provides more flexibility in terms of controlling the loop's execution based on dynamic conditions.

FUNCTIONS – REQUIREMENT

- Functions promote code modularity and reusability.
- Imagine you have a task that needs to be performed multiple times within a program.
- Instead of duplicating the same code at various places, you can define a function once and call it whenever you need that task.
- This reduces redundancy and makes the code easier to manage and maintain.



BENEFITS OF FUNCTIONS

- Modularity: Functions break down complex tasks into manageable components
- Reusability: Functions can be used multiple times without rewriting code
- Readability: Functions with meaningful names enhance code understanding
- Debugging: Isolating functions eases troubleshooting and issue fixing
- Abstraction: Functions simplify complex processes behind a user-friendly interface
- Collaboration: Team members can work on different functions concurrently
- Maintenance: Changes made in a function automatically apply wherever it's used

EXAMPLE

Consider a function named `calculate_total` that takes two numbers as input (parameters), adds them together, and then produces the sum as the output. Here's how it works:

```
1  def calculate_total(a, b): # Parameters: a and b
2      total = a + b         # Task: Addition
3      return total          # Output: Sum of a and b
4
5  result = calculate_total(5, 7) # Calling the function with inputs 5 and 7
6  print(result) # Output: 12
```



BUILT IN FUNCTIONS

len(): Calculates the length of a sequence or collection

```
1 string_length = len("Hello, World!") # Output: 13
2 list_length = len([1, 2, 3, 4, 5]) # Output: 5
```



sum(): Adds up the elements in an iterable (list, tuple, and so on)

```
1 total = sum([10, 20, 30, 40, 50]) # Output: 150
```



max(): Returns the maximum value in an iterable

```
1 highest = max([5, 12, 8, 23, 16]) # Output: 23
```



min(): Returns the minimum value in an iterable

```
1 lowest = min([5, 12, 8, 23, 16]) # Output: 5
```



DEFINING OWN FUNCTION

1. Use `def` followed by the function name and parentheses

Here is the syntax to define a function:

```
1 def function_name():  
2     pass
```

A `"pass"` statement in a programming function is a placeholder or a no-op (no operation) statement. Use it when you want to define a function or a code block syntactically but do not want to specify any functionality or implementation at that moment.

- **Placeholder:** `"pass"` acts as a temporary placeholder for future code that you intend to write within a function or a code block.
- **Syntax Requirement:** In many programming languages like Python, using `"pass"` is necessary when you define a function or a conditional block. It ensures that the code remains syntactically correct, even if it doesn't do anything yet.
- **No Operation:** `"pass"` itself doesn't perform any meaningful action. When the interpreter encounters `"pass"`, it simply moves on to the next statement without executing any code.

DOCS STRINGS

- Docstrings explain what a function does
- Placed inside triple quotes under the function definition
- Helps other developers understand your function

Example:

```
1  def multiply(a, b):
2      """
3      This function multiplies two numbers.
4      Input: a (number), b (number)
5      Output: Product of a and b
6      """
7      print(a * b)
8  multiply(2,6)
```

RETURN STATEMENT

Return statement

- Return gives back a value from a function
- Ends the function's execution and sends the result
- A function can return various types of data

Example:

```
1 def add(a, b):  
2     return a + b  
3  
4 sum_result = add(3, 5) # sum_result gets the value 8
```





SCOPES

- **Global Scope:** Variables defined outside functions; accessible everywhere
- **Local Scope:** Variables inside functions; only usable within that function

GLOBAL VARIABLE

python

Copy Edit

Global variable

x = 10

def my_function():

Local variable

y = 5

print("Inside function - Local variable y:", y)

print("Inside function - Global variable x:", x) *# Accessing global variable inside function*

Calling the function

my_function()

Trying to access local variable outside function (This will cause an error)

print("Outside function - Local variable y:", y) *# This will raise a NameError*